INGENIERÍA DE SOFTWARE

GUÍA DE ESTUDIO Y LABORATORIO

ISO/IEC 25010:2023

CALIDAD DEL SOFTWARE

QUE SE PUEDE DEMOSTRAR

Semana 10 · Adecuación funcional

Semana 11 · Compatibilidad

Semana 12 · Capacidad de interacción

Semana 13 · Informe APF3 y versión 2

REVISADO Y APROBADO POR

Mg. Ing. Huamani Uriarte Enrique Lee

CURSO INTEGRADOR II: SOFTWARE · UTP · EDICIÓN 2026

Presentación editorial

Este manual es una guía de aprendizaje y de trabajo para la **Unidad 3 del Curso Integrador II: Software (100000S12F)**. Se organiza por las semanas 10, 11 y 12 y culmina con la preparación del **APF3 de la semana 13**, incluyendo la demostración de la versión 2 de la solución. Su finalidad es que el estudiante comprenda los conceptos de calidad y los convierta en pruebas que cualquier evaluador pueda reproducir.

La obra retoma el sílabo del ciclo 2026-2 y utiliza como referencia general el modelo de calidad del producto **ISO/IEC 25010:2023**. Explica en lenguaje docente los conceptos pertinentes al curso, sin reproducir el texto completo de la norma. Las fórmulas, umbrales, rúbricas y plantillas que aquí aparecen son **instrumentos académicos propuestos**, no requisitos numéricos fijados por ISO.

NOTA DE INTEGRIDAD ACADÉMICA

La aprobación editorial solicitada en esta portada identifica al responsable académico señalado para el material. **No equivale a una certificación de conformidad ISO**, ni sustituye la evaluación institucional del proyecto. Los datos del caso de ejemplo son ficticios y deben sustituirse por evidencias reales de cada grupo.

Créditos académicos

Campo	Información
Título	Calidad del software que se puede demostrar. Manual integral de estudio, laboratorio e informe APF3.
Curso	Curso Integrador II: Software · Ingeniería de Software · UTP.
Edición	Primera edición de uso didáctico, noviembre de 2026.
Responsable de revisión y aprobación	**Mg. Ing. Huamani Uriarte Enrique Lee**.
Alcance	Semanas 10, 11, 12 y evaluación de semana 13.
Lectura	Material formativo de elaboración original; consultar las normas oficiales y referencias al final.

Contrato de aprendizaje

El libro funciona como un instructor: primero explica por qué importa un criterio, después presenta un ejemplo de un sistema ficticio, propone una práctica observable y termina indicando qué documento, captura o resultado debe conservarse para el informe. A lo largo de la lectura, el estudiante completará un **portafolio de evidencias** para sustentar decisiones, no solo para llenar páginas.

- No se exige dominar un lenguaje concreto; puedes adaptar los ejercicios a Java, Python, JavaScript, PHP, .NET o al stack de tu proyecto.
- Trabaja con datos sintéticos; evita mostrar nombres, documentos, correos, tokens y credenciales de personas reales.
- Registra pruebas satisfactorias, fallidas, bloqueadas y no ejecutadas; ninguna se debe ocultar.
- Conserva condiciones de ejecución: fecha, versión, entorno, herramientas, responsables y criterios previos.

Índice general

Selecciona un capítulo para navegar desde Word. Las páginas corresponden a la versión PDF incluida.

Presentación editorial....2
Ruta de aprendizaje y mapeo de sílabo....4
SEMANA 10 | ADECUACIÓN FUNCIONAL....5
01. Comprender la calidad de software y la norma....6
02. Requisitos, criterios y trazabilidad....8
03. Completitud, corrección y pertinencia....10
04. Verificación, validación y pruebas funcionales....12
SEMANA 11 | COMPATIBILIDAD....14
05. Interoperabilidad: intercambiar y usar información....15
06. Contratos HTTP, API REST y fallos controlados....17
07. Coexistencia y diseño experimental....19
08. Laboratorio integrado de compatibilidad....21
SEMANA 12 | CAPACIDAD DE INTERACCIÓN....23
09. El modelo de capacidad de interacción....24
10. Aprendibilidad y protocolos con usuarios....26
11. Protección contra errores y asistencia al usuario....28
12. Involucración, inclusión y autodescriptividad....30
13. Laboratorio de interacción y mejora de interfaz....32
SEMANA 13 | APF3 · EVALUACIÓN Y VERSIÓN 2....34
14. El APF3 y sus productos obligatorios....35
15. Metodología, indicadores y decisión de calidad....37
16. Redacción completa del informe técnico APF3....39
17. Miniinforme resuelto: caso ficticio de citas....41
18. Sustentación, rúbrica y checklist final....43
RECURSOS COMPLEMENTARIOS....45
Anexos A–J · Plantillas editables....45
Banco de actividades y ejercicios....48
Guía de instalación del laboratorio....49
Prompts para el estudio y revisión....50
Glosario de términos....51
Referencias y lecturas....52

Ruta de aprendizaje y correspondencia con el sílabo



Figura. Progresión didáctica de las semanas 10–13 y su conexión con el informe APF3.

Semana / sesiones	Tema oficial del sílabo	Resultado esperado
10 · sesiones 19 y 20	Calidad orientada a la funcionalidad; completitud, corrección e idoneidad; verificación y validación.	Requisitos trazados, plan de pruebas funcionales y resultados sustentados.
11 · sesiones 21 y 22	Compatibilidad; interoperabilidad y coexistencia.	Contrato de integración, pruebas de API y experimento de coexistencia.
12 · sesiones 23 y 24	Capacidad de interacción; facilidad de aprendizaje, protección contra errores, asistencia al usuario e involucración.	Protocolo con usuarios, resultados de tareas y mejora de interfaz con evidencias.
13 · sesiones 25 y 26	Recapitulación; evidencias de funcionalidad, compatibilidad e interacción; despliegue de versión 2; APF3.	Informe técnico integrado, anexos, versión 2 y defensa de resultados.

Cómo estudiar de forma autónoma

Secuencia de 5 movimientos

1. **Leer y comprender:** explica en tus palabras el atributo de calidad antes de seleccionar una herramienta.
2. **Traducir a criterios:** define qué debe ocurrir y cómo comprobarás si realmente ocurrió.
3. **Ejecutar la actividad:** usa tu proyecto o el laboratorio local, conserva parámetros y versión.
4. **Interpretar:** explica por qué un resultado apoya o cuestiona la calidad evaluada.
5. **Documentar:** conecta código de requisito, prueba, resultado, hallazgo y evidencia.

Qué no es el alcance de esta unidad

La norma de producto incluye **nueve características**, pero el sílabo de esta unidad solicita trabajar directamente tres: **adecuación funcional, compatibilidad y capacidad de interacción**. Rendimiento, flexibilidad, mantenibilidad y confiabilidad se desarrollarán en la Unidad 4. Se pueden recoger valores de tiempo y recursos aquí como evidencia de coexistencia, sin declarar que se ha evaluado integralmente toda la característica de eficiencia del desempeño.

PARTE I | SEMANA 10

Adecuación funcional

De la necesidad empresarial a una prueba que demuestra que la solución funciona.

UNIDAD 3 · CALIDAD DE SOFTWARE

PREGUNTA GUÍA

¿El sistema ofrece las funciones correctas, completas y apropiadas para resolver el problema?

CAPÍTULO 01 / SEMANA 10

Comprender la calidad de software y la norma

LO QUE CONSEGUIRÁS

Diferenciar calidad, funcionamiento, conformidad y evidencia; identificar el alcance real de la ISO/IEC 25010:2023.

1.1 El software puede encender y aun así fallar

Una aplicación de citas permite pulsar “Reservar” y muestra “Operación exitosa”. ¿Es suficiente? No. Tal vez el horario estaba ocupado, la reserva no se guardó o el usuario recibió una confirmación engañosa. Por eso **software que funciona** no significa necesariamente **software de calidad**. La calidad exige observar el comportamiento frente a necesidades y condiciones concretas.

El sílabo ubica esta unidad después de la planificación, el front-end, el back-end, la seguridad y el primer despliegue. Ese orden tiene una lógica: durante las semanas 10–13 el equipo debe demostrar que la solución desarrollada es útil, compatible con sistemas externos y comprensible para quienes la utilizan.

1.2 El modelo de calidad: una referencia, no una medalla

ISO/IEC 25010:2023, segunda edición de noviembre de 2023, presenta un modelo de **nueve características** para describir y evaluar la calidad de productos TIC y software. Es una referencia para formular requisitos y criterios de evaluación; por sí sola **no entrega certificados ni establece un porcentaje universal de aprobación** para cualquier aplicación. El equipo debe definir metas específicas y justificarlas en su contexto.

Pregunta del evaluador	Respuesta que no basta	Respuesta que demuestra
¿La función está completa?	“Ya está programada”.	“RF-04 exige tres acciones; se comprobaron las tres en CP-04A, B y C”.
¿La integración funciona?	“Tenemos una API”.	“Se intercambiaron datos válidos, se rechazó el contrato incorrecto y se trató el fallo 503”.
¿La interfaz es fácil?	“Se ve moderna”.	“4 de 5 participantes completaron la tarea sin asistencia; se registraron errores”.

1.3 Tres características pertinentes para la unidad

Adecuación funcional: alcance de funciones, resultados correctos y utilidad para realizar tareas. **Compatibilidad:** intercambio y uso de información con otros productos, más coexistencia al compartir un entorno. **Capacidad de interacción:** recursos del producto que facilitan comprenderlo, aprenderlo, utilizarlo y prevenir errores. Estas etiquetas corresponden al enfoque actualizado de ISO/IEC 25010:2023.

UNA PRECISIÓN HISTÓRICA

Materiales basados en ISO/IEC 25010:2011 pueden usar “usabilidad” como nombre principal. En la edición 2023 la característica del producto se denomina **capacidad de interacción (interaction capability)**. Como el sílabo menciona “Usability”, el manual conserva la equivalencia pedagógica sin confundir versiones.

1.4 Las cinco piezas de una evaluación defendible

- **Requisito:** comportamiento o atributo esperado, identificable y comprobable.

- **Criterio de aceptación:** condición verificable para decidir si se cumple.
- **Procedimiento:** pasos, datos, herramientas y entorno de la prueba.
- **Resultado:** observación obtenida, incluso cuando contradice lo esperado.
- **Evidencia:** registro suficientemente trazable para que otra persona revise la conclusión.

Comprueba que aprendiste

1. ¿Por qué un botón que muestra “Éxito” no demuestra corrección funcional?
2. ¿Puede un sistema tener un 100 % de pruebas aprobadas y aun tener requisitos sin probar?

APLÍCALO A TU PROYECTO

Describe una función de tu proyecto que aparente trabajar pero pudiera producir resultados incorrectos. Redacta un ejemplo de evidencia verificable.

EVIDENCIA PARA LA SEMANA 13

Una ficha con requisito, riesgo de error, procedimiento propuesto y resultado esperado.

CAPÍTULO 02 / SEMANA 10

Requisitos, criterios y trazabilidad

LO QUE CONSEGUIRÁS

Redactar requisitos verificables y construir el hilo requisito → caso → resultado → evidencia.

2.1 Del problema real al requisito comprobable

En un centro de atención, los colaboradores anotan citas manualmente y se producen reservas duplicadas. La solución propone gestionar horarios desde una web. El enunciado “el sistema será eficiente y amigable” es ambiguo: no define qué probar. Conviene descomponerlo en comportamientos y establecer resultados observables.

ID	Requisito verificable	Criterio de aceptación inicial
RF-01	Registrar una cita indicando nombre ficticio, fecha y horario.	Ante datos válidos, responde HTTP 201 y devuelve un identificador.
RF-02	Evitar reservas duplicadas del mismo horario.	Ante duplicidad, responde HTTP 409 y no crea otro registro.
RF-03	Consultar horarios desde proveedor externo.	Solicita disponibilidad y presenta campos conforme al contrato.
RF-04	Guiar al usuario si omite un campo obligatorio.	Identifica el campo y sugiere cómo corregirlo sin perder datos válidos.

2.2 Del requisito al diseño de pruebas

Cada requisito debe contar con al menos una prueba positiva cuando corresponda y pruebas negativas para condiciones importantes. En RF-02, un caso positivo verifica que la primera cita se registre; otro caso comprueba que la repetición sea rechazada. Cambiar únicamente la entrada o el contexto puede revelar fallos invisibles en una demostración preparada.

Caso	Precondición	Acción	Resultado esperado
CP-F01	Servidor y proveedor disponibles.	POST /api/citas con datos válidos.	201 + id y reserva guardada.
CP-F02	Horario ya reservado.	Repetir fecha y hora.	409; no aumenta el número de citas.
CP-F03	Nombre vacío.	Enviar nombre = "".	400 con mensaje de corrección.
CP-I01	Proveedor detenido.	GET /api/disponibilidad.	503 controlado, sin error interno expuesto.

2.3 Matriz bidireccional de trazabilidad

La matriz responde dos preguntas: **¿qué prueba verifica cada requisito?** y **¿por qué ejecutamos cada prueba?**. Es bidireccional porque permite detectar requisitos sin pruebas y pruebas sin propósito. Debe mantenerse actualizada cuando cambien el alcance o la versión de software.

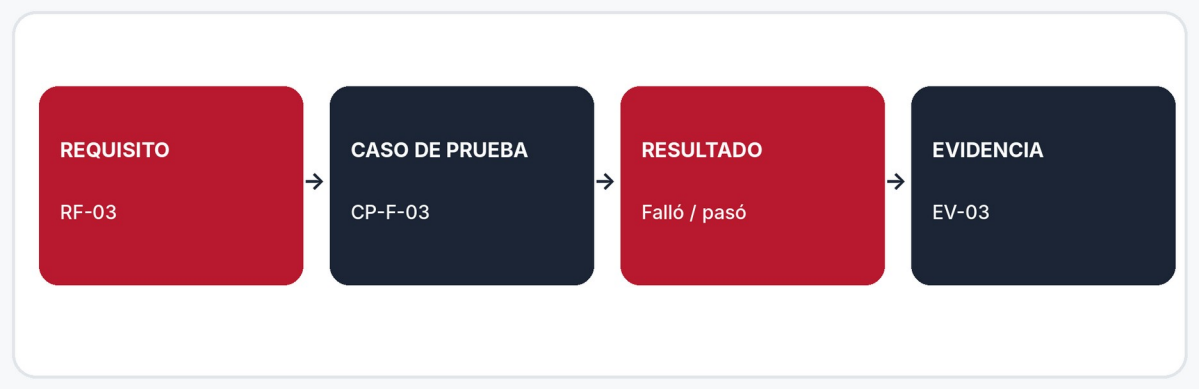


Figura. Una sola captura no reemplaza la cadena de trazabilidad completa.

2.4 Ejercicio de redacción de criterios

Enunciado inicial: “la aplicación debe ser rápida”. Versión comprobable, para un experimento acordado: “en el entorno de laboratorio y para la consulta de disponibilidad con 20 solicitudes consecutivas, el percentil 95 de tiempo de respuesta no superará 600 ms”. El valor **600 ms es una meta didáctica propuesta**, no una cifra exigida por la ISO; debe ser razonable según infraestructura, usuarios y objetivos del proyecto.

EVITA ESTE ERROR

No se deben establecer umbrales después de conocer los datos solamente para aparentar cumplimiento. Define meta, procedimiento, entorno y fórmula antes de ejecutar las pruebas.

Comprueba que aprendiste

1. ¿Qué diferencia hay entre requisito, criterio y caso de prueba?
2. ¿Cómo identificarías un requisito que no tiene evidencia?

APLÍCALO A TU PROYECTO

Selecciona seis requisitos reales del proyecto y redáctalos con ID, rol, condición y aceptación medible.

EVIDENCIA PARA LA SEMANA 13

Matriz RF–CP–EV de al menos seis requisitos, con estado inicial y responsable.

CAPÍTULO 03 / SEMANA 10

Completitud, corrección y pertinencia

LO QUE CONSEGUIRÁS

Evaluar y separar las tres subcaracterísticas funcionales con medidas justificadas.

3.1 Completitud funcional

Pregunta: **¿están disponibles todas las funciones exigidas para realizar las tareas previstas?** Si se comprometieron ocho funciones y solo se implementaron seis, hay una brecha de alcance; que las seis disponibles funcionen bien no resuelve la ausencia de las dos restantes. Es indispensable diferenciar "desarrollado", "probado" y "aceptado".

Indicador académico de cobertura: C = funciones requeridas verificadas / funciones requeridas acordadas × 100. Ejemplo ilustrativo: seis funciones verificadas de ocho comprometidas → C = 6/8 × 100 = **75 %**. La definición de "función" y el denominador deben quedar documentados. No es una fórmula obligatoria de la ISO.

3.2 Corrección funcional

Pregunta: **¿las funciones entregan resultados exactos y válidos?** Una reserva que muestra la hora incorrecta, modifica la fecha o permite duplicados falla en corrección aunque exista la pantalla y se pueda hacer clic. Prueba entradas normales, inválidas, valores límite y estados de conflicto.

Ejemplo didáctico: de 12 casos de corrección, nueve pasaron, dos fallaron y uno quedó bloqueado. La tasa de aprobación entre ejecutados es 9/11 = 81,8 %. El caso bloqueado se informa aparte; nunca se contabiliza como aprobado. La tasa de pruebas aprobadas no debe confundirse con cobertura total ni conformidad integral.

3.3 Pertinencia o idoneidad funcional

Pregunta: **¿las funciones ayudan de verdad a lograr el objetivo sin pasos innecesarios?** Un operador debe reservar una cita en una pantalla clara, no abrir seis módulos ni reingresar información. Esta subcaracterística se relaciona con la tarea y puede analizarse combinando mapa de proceso, observación y comparación de pasos efectivos.

Subcaracterística	Pregunta guía	Ejemplo de medida / evidencia
Completitud	¿Falta alguna función comprometida?	8 RF acordados → 6 implementados y probados; matriz de cobertura.
Corrección	¿Produce el dato o estado esperado?	Pruebas de validación, duplicado y resultados API.
Pertinencia	¿Ayuda a realizar la tarea?	Comparación de pasos del proceso y observación de usuarios.

3.4 Conclusión de ejemplo

Conclusión defendible: "El módulo alcanzó cobertura del 75 % de funciones acordadas. Las seis funciones probadas respondieron conforme a sus criterios en la ejecución documentada, pero persisten dos pendientes; por ello no es válido afirmar completitud general. El proceso de registro aún requiere cinco pasos y se propone simplificarlo a tres".

Comprueba que aprendiste

1. Si 8/8 requisitos están implementados, ¿por qué aún pueden existir errores?
2. ¿Por qué “pertinencia” no es igual a belleza visual?

APLÍCALO A TU PROYECTO

Calcula cobertura de funciones y aprobación de pruebas con datos originales de tu software. Separa tres resultados y sus limitaciones.

EVIDENCIA PARA LA SEMANA 13

Tres párrafos analíticos, uno por subcaracterística, con cálculo o prueba propia.

CAPÍTULO 04 / SEMANA 10

Verificación, validación y pruebas funcionales

LO QUE CONSEGUIRÁS

Diseñar casos positivos, negativos y de borde, registrar defectos y explicar las decisiones.

4.1 La distinción que nunca se debe olvidar

Verificación pregunta: "¿el producto está construido según lo especificado?". **Validación** pregunta: "¿el producto resuelve la necesidad de uso prevista?". Por ejemplo, verificar RF-01 es constatar que el servidor responde 201 bajo condiciones acordadas; validar es comprobar con el encargado que ese proceso de reserva realmente satisface su forma de trabajar.

4.2 Técnicas útiles para empezar

- **Partición de equivalencia:** divide entradas válidas e inválidas representativas.
- **Valores límite:** prueba justo por debajo, en y justo por encima de un límite.
- **Tabla de decisión:** combina condiciones relevantes y su acción correspondiente.
- **Transición de estados:** verifica cómo cambia una cita entre libre, reservada y cancelada, si ese flujo existe.
- **Regresión:** tras corregir un defecto, vuelve a ejecutar pruebas relacionadas.

ID	Entrada	Resultado esperado	Categoría
CP-01	Nombre = "Ana"; fecha válida; hora libre.	Reserva creada · HTTP 201.	Positiva
CP-02	Nombre = "".	Rechazo explicativo · HTTP 400.	Negativa
CP-03	Nombre = "A".	Rechazo: mínimo dos caracteres.	Borde inferior
CP-04	Mismo día y horario de cita previa.	Conflicto · HTTP 409.	Estado / regla
CP-05	Proveedor no responde.	Fallo controlado · HTTP 503.	Resiliencia

4.3 Una ficha que otro equipo puede repetir

La ficha mínima reúne: **ID, versión del sistema, requisito vinculado, objetivo, entorno, precondiciones, datos, pasos, resultado esperado, resultado obtenido, estado, evidencia, fecha y responsable**. Si solo se adjunta una captura con la palabra "Éxito", el evaluador no sabrá cómo se obtuvo ni si representa una prueba relevante.

CRITERIO DE DECISIÓN

Estados: **APROBADA** (se cumplió el criterio), **FALLIDA** (diferencia reproducible), **BLOQUEADA** (dependencia impide ejecutarla), **NO EJECUTADA** (planificada sin correr). Registra la justificación cuando aplique.

4.4 Laboratorio guiado: pruebas de la semana 10

Ejecución con el paquete de laboratorio adjunto

1. Crear la carpeta del laboratorio y abrirla en VS Code. En una terminal ejecutar `python -m venv .venv` y activar el entorno virtual.
2. Instalar dependencias con `python -m pip install -r requirements.txt`. Las bibliotecas se instalarán en el entorno.
3. Ejecutar `python -m unittest test_sistema.py -v`. El proveedor externo se simula en las pruebas; no es necesario iniciarlo.
4. Anotar cuántas pruebas pasan o fallan, tomar evidencia con terminal, fecha, versión e identificador del test.
5. Alterar temporalmente una regla de validación (solo en copia de laboratorio), repetir las pruebas y explicar qué fallo detectaron; revertir el cambio.

```
python -m unittest test_sistema.py -v
# Esperado con el código original: 6 pruebas OK.
# La evidencia real es la salida obtenida en el equipo del estudiante.
```

Comprueba que aprendiste

1. ¿Cuándo un test debe marcarse BLOQUEADO y no FALLIDO?
2. ¿Qué demuestra una prueba de regresión después de una corrección?

APLÍCALO A TU PROYECTO

Diseña ocho casos para una funcionalidad de tu proyecto: tres positivos, tres negativos y dos de límite o transición.

EVIDENCIA PARA LA SEMANA 13

Ocho fichas o filas de ejecución, salida real, captura de ejecución y síntesis de fallos.

PARTE II | SEMANA 11

Compatibilidad del software

Interoperabilidad y coexistencia: dos ideas diferentes, dos diseños de prueba.

UNIDAD 3 · CALIDAD DE SOFTWARE

PREGUNTA GUÍA

¿Nuestro sistema intercambia datos confiables con otros y funciona bien en un entorno compartido?

CAPÍTULO 05 / SEMANA 11

Interoperabilidad: intercambiar y usar información

LO QUE CONSEGUIRÁS

Distinguir integración de interoperabilidad y definir contratos observables entre sistemas.

5.1 Integrarse no es solo “llamar a una URL”

Un software de citas puede depender de un proveedor de horarios. Si realiza una petición HTTP pero malinterpreta el campo recibido, no ha demostrado interoperabilidad. **Interoperabilidad** significa intercambiar información y utilizar correctamente la información intercambiada. Es esencial documentar formato, significado de campos, autenticación cuando exista, versiones y errores.



Figura. Arquitectura didáctica: navegador, sistema principal y servicio externo ficticio.

5.2 El contrato de la integración

Elemento	Contrato didáctico del proveedor
Operación	GET /v1/disponibilidad?fecha=AAAA-MM-DD
Respuesta 200	JSON con fecha y arreglo horarios.
Error 400	Parámetro de fecha faltante o inválido.
Condición de éxito	El sistema interpreta horarios, ofrece opciones y conserva el significado.
Fallo externo	Respuesta controlada de la aplicación y registro técnico sin exponer secretos.

```
GET /v1/disponibilidad?fecha=2026-11-05
HTTP/1.1 200 OK
Content-Type: application/json

{"fecha":"2026-11-05","horarios":["09:00","10:00","11:00"]}
```

5.3 Interoperabilidad observada frente a integración declarada

El equipo debe probar: petición válida; respuesta estructuralmente correcta; significado de datos; campos faltantes; códigos 4xx y 5xx; fallo por tiempo de espera; recuperación y comunicación adecuada al usuario. Un único “200 OK” no demuestra que el sistema utilice los datos ni que responda de forma robusta ante variaciones.

5.4 Cuida el límite de seguridad

El laboratorio usa nombres inventados y HTTP local para evitar manejar información de personas reales. Un entorno productivo requiere considerar HTTPS, credenciales protegidas, autenticación/autorización y minimización de datos. La guía OWASP API Security Top 10 sirve como recurso complementario de seguridad, no como sustituto del tema de interoperabilidad.

Comprueba que aprendiste

1. ¿Qué diferencia hay entre recibir JSON y usar correctamente su contenido?
2. ¿Por qué un 200 OK no basta para demostrar éxito de una reserva?

APLÍCALO A TU PROYECTO

Dibuja los sistemas que integrará tu proyecto. Marca origen, destino, datos, condición de éxito y modo de fallo.

EVIDENCIA PARA LA SEMANA 13

Diagrama de arquitectura, ficha de contrato y dos pruebas de interpretación de datos.

CAPÍTULO 06 / SEMANA 11

Contratos HTTP, API REST y fallos controlados

LO QUE CONSEGUIRÁS

Realizar pruebas reproducibles con herramientas HTTP y documentar los errores esperados.

6.1 Qué debes entender de HTTP antes de usar Postman

El **método** expresa la operación, la **ruta** identifica el recurso, el **cuerpo** envía datos cuando corresponde, la **respuesta** devuelve un estado y una representación. En el laboratorio: GET consulta disponibilidad; POST crea citas. Los códigos 201 (creación), 400 (entrada inválida), 409 (conflicto), 502 (respuesta incorrecta del proveedor) y 503 (dependencia indisponible) tienen significados distintos.

Prueba	Petición	Verificación
I-01	GET /api/disponibilidad?fecha=2026-11-05	200; fecha correcta y arreglo de horarios.
I-02	GET /api/disponibilidad?fecha=mal	400; indicación de formato.
I-03	POST /api/citas con JSON válido	201; id y datos persistidos en memoria.
I-04	Detener proveedor y consultar.	503 controlado; sin excepción visible.

6.2 Prueba automatizada en Postman

En Postman crea una solicitud GET a `http://127.0.0.1:5000/api/disponibilidad?fecha=2026-11-05`. Después abre **Scripts > Post-response** e incorpora aserciones sobre el resultado. Si el proveedor se ha iniciado correctamente, la expectativa didáctica es que ambos tests pasen.

```
pm.test("Responde 200", () => {
  pm.response.to.have.status(200);
});
pm.test("La respuesta incluye horarios", () => {
  const data = pm.response.json();
  pm.expect(data).to.have.property("horarios");
  pm.expect(data.horarios).to.be.an("array");
});
```

6.3 Pruebas negativas deliberadas

Detén `proveedor.py` sin cerrar `sistema.py`; repite GET disponibilidad. Espera un **503** con mensaje controlado. No adjuntes solamente el resultado: registra precondición, hora, puerto, versión, expectativa y observación. Este ejercicio muestra que una integración también se evalúa en sus condiciones de fallo.

NO CONFUNDAS LOS CÓDIGOS

Un código HTTP correcto puede coexistir con datos incorrectos; y algunos fallos bien controlados (400, 409, 503) son el **resultado correcto** de una prueba negativa. La prueba aprueba cuando coincide con su criterio, no cuando todo devuelve 200.

Comprueba que aprendiste

1. ¿Por qué una prueba que espera 409 puede aprobar?
2. ¿Qué campo de un contrato se omite con frecuencia y luego produce errores de integración?

APLÍCALO A TU PROYECTO

Documenta al menos cinco escenarios, incluyendo éxito, dato inválido, contrato incorrecto y falla del servicio externo.

EVIDENCIA PARA LA SEMANA 13

Colección de pruebas o equivalente, capturas/exports y matriz de interoperabilidad.

CAPÍTULO 07 / SEMANA 11

Coexistencia y diseño experimental

LO QUE CONSEGUIRÁS

Separar interoperabilidad de coexistencia y comparar línea base frente a recursos compartidos.

7.1 La diferencia imprescindible

Coexistencia es que productos compartan recursos y ambiente sin deteriorarse de forma perjudicial. No implica necesariamente intercambiar datos. Un sistema de citas y otro proceso en el mismo equipo pueden coexistir bien aunque no se comuniquen. Por contraste, dos APIs pueden interoperar en máquinas diferentes sin haber demostrado coexistencia.

7.2 Diseña un experimento antes de medir

Protocolo de dos condiciones

1. Define el entorno: CPU, RAM, sistema operativo, navegador, versión de software, red y configuración.
2. Selecciona la operación objetivo (por ejemplo, consulta de estado de salud) y define un criterio medible previo.
3. **Condición A:** mide al sistema principal sin la segunda carga; repite el ensayo al menos en dos rondas.
4. **Condición B:** introduce un proceso local moderado que comparta CPU y repite mediciones equivalentes.
5. Registra latencia por solicitud, fallos, mediana, P95 y uso de CPU si cuentas con una herramienta fiable.
6. Compara ambos resultados y explica variabilidad, interferencias y limitaciones.

7.3 Medidas y cálculos interpretables

El **percentil 95 (P95)** es un valor de referencia por debajo del cual se encuentra aproximadamente el 95 % de observaciones. Para muestras pequeñas depende de la definición de percentil utilizada; escribe tu método de cálculo. El **cambio relativo** puede calcularse como $\Delta\% = (P95_B - P95_A) / P95_A \times 100$. Un cambio positivo puede señalar degradación; su importancia debe contrastarse con el objetivo acordado y la variabilidad.

Condición (datos ficticios)	P95 (ms)	Errores / solicitudes	Lectura prudente
A: sin carga adicional	220	0 / 40	Línea base de referencia.
B: con carga local	264	0 / 40	$\Delta = (264-220)/220 = 20\ \%$. Degradación observada.

No concluyas “aprueba ISO” solo por el 20 %. Si se pactó antes del ensayo un máximo de 15 %, ese criterio no se alcanzó; si no se pactó, hay una observación descriptiva. Además, una única serie de 40 solicitudes no permite generalizaciones amplias.

7.4 Amenazas a la validez

- Cambios de temperatura, actualizaciones del sistema, procesos ocultos, caché y red pueden alterar mediciones.
- Solicitudes concurrentes a un servidor de desarrollo no representan necesariamente producción.
- El tamaño de muestra, la carga inducida y el sistema operativo condicionan el resultado.
- No sometas a carga aplicaciones ajenas, servicios externos o sistemas productivos sin autorización.

Comprueba que aprendiste

1. ¿Puede existir coexistencia sin intercambio de datos?
2. ¿Por qué se necesita una línea base comparable?

APLÍCALO A TU PROYECTO

Diseña dos condiciones replicables y declara criterio antes de medir. Calcula Δ% con datos reales de laboratorio.

EVIDENCIA PARA LA SEMANA 13

Ficha del entorno, tabla A/B, medidas de dispersión, capturas y discusión de límites.

CAPÍTULO 08 / SEMANA 11

Laboratorio integrado de compatibilidad

LO QUE CONSEGUIRÁS

Ejecutar integración local y una comparación controlada de coexistencia.

8.1 Preparar dos servicios

Paso a paso desde cero

1. Descarga y descomprime el **laboratorio complementario**. Abre dos terminales en esa carpeta.
2. En terminal 1, ejecuta `python proveedor.py`. Debe indicar dirección local y puerto 5001.
3. En terminal 2, ejecuta `python sistema.py`. Debe indicar dirección local y puerto 5000.
4. En navegador abre `http://127.0.0.1:5000/`. Usa exclusivamente nombres ficticios.
5. Visita la ruta `/api/disponibilidad?fecha=2026-11-05` del puerto 5000 y guarda la respuesta de integración.
6. Detén el proveedor y ejecuta la misma petición: registra si el sistema devuelve 503 controlado.

8.2 Correr pruebas de coexistencia

```
# Con sistema.py en marcha, primero sin carga:
python medir_coexistencia.py
# Luego, en una segunda terminal inicia carga CPU local:
python carga_suave.py
# Y mientras corre, repite la medición:
python medir_coexistencia.py
```

Realiza varias rondas para disminuir la dependencia de una ejecución aislada. En cada una anota **fecha/hora, hardware, condición, número de solicitudes, errores, mediana y P95**. La carga suave dura 12 segundos por ejecución; si la medición queda fuera de ese intervalo, repítela documentando la limitación. Este ensayo es introductorio y no sustituye una prueba de desempeño formal.

8.3 Producto mínimo del laboratorio

Archivo de evidencia	Contenido exigido
EV-I-01	Petición y respuesta válida del proveedor interpretada por el sistema.
EV-I-02	Prueba con dependencia detenida y resultado controlado.
EV-C-01	Línea base de coexistencia, entorno y datos crudos.
EV-C-02	Condición compartida, resultados y cálculo de variación.

Comprueba que aprendiste

1. ¿Qué harías si ambos servicios corren pero `/api/disponibilidad` responde 503?
2. ¿Cuál es la diferencia entre salida simulada y prueba ejecutada?

APLÍCALO A TU PROYECTO

Repite el laboratorio o traslada el protocolo a tu stack real sin forzar una arquitectura innecesaria.

EVIDENCIA PARA LA SEMANA 13

Cuatro evidencias numeradas, 2 fichas de integración, 1 experimento de coexistencia y hallazgos.

PARTE III | SEMANA 12

Capacidad de interacción

Aprender, proteger al usuario, asistirlo y evaluar la experiencia mediante tareas observables.

UNIDAD 3 · CALIDAD DE SOFTWARE

PREGUNTA GUÍA

¿Una persona puede aprender, completar tareas y recuperarse de errores sin quedar bloqueada?

CAPÍTULO 09 / SEMANA 12

El modelo de capacidad de interacción

LO QUE CONSEGUIRÁS

Identificar la característica de la edición 2023, sus subcaracterísticas y el alcance del sílabo.

9.1 Del interfaz “bonito” a la interacción demostrable

Una pantalla atractiva no garantiza que las personas encuentren un botón, entiendan el mensaje de un error o sepan terminar una tarea. La **capacidad de interacción** evalúa propiedades del producto que permiten a los usuarios interactuar con él en un contexto especificado. No debe inferirse directamente por opiniones del equipo desarrollador.

9.2 Ocho subcaracterísticas del modelo 2023

Subcaracterística	Explicación didáctica	Ejemplo observable
Reconocibilidad de la adecuación	El usuario comprende para qué le sirve el sistema.	Identifica cuál es la función de reservar.
Aprendibilidad (facilidad de aprendizaje)	Aprende funciones determinadas en un tiempo objetivo.	Realiza reserva inicial con instrucciones mínimas.
Operabilidad	Puede controlar el producto sin dificultad indebida.	Navega y cambia información con facilidad.
Protección contra errores del usuario	El sistema previene o reduce errores operativos.	Evita dobles envíos y marca campos inválidos.
Involucración del usuario (user engagement)	La presentación invita a continuar interactuando.	Encuesta contextual + observación, sin confundir con adicción.
Inclusividad	Funciona para personas de contextos diversos.	Lenguaje claro y pruebas con perfiles variados.
Asistencia al usuario	Ofrece ayuda para completar objetivos.	Mensaje concreto explica cómo corregir una fecha.
Autodescriptividad	Permite comprender controles sin depender excesivamente de recursos externos.	Etiquetas visibles, pasos y mensajes contextualizados.

El **sílabo enfatiza** aprendibilidad, protección contra errores, asistencia al usuario e involucración. Las otras cuatro subcaracterísticas se incluyen para que el estudiante comprenda el modelo actualizado y pueda justificar su alcance sin fingir que las evaluó integralmente.

9.3 Una observación no es una conclusión automática

“A cuatro personas les gustó la pantalla” no prueba necesariamente facilidad de aprendizaje; puede ser una valoración visual. Para valorar aprendizaje se necesita una tarea nueva, un criterio de éxito, un tiempo registrado y una definición de ayuda permitida. Para la protección contra errores, se observa si previene, identifica o facilita recuperar el error.

Comprueba que aprendiste

1. ¿Qué diferencia hay entre aprendibilidad e involucración?
2. ¿Cuál de las ocho subcaracterísticas estudia si la interfaz se explica por sí misma?

APLÍCALO A TU PROYECTO

Selecciona cuatro subcaracterísticas aplicables a tu proyecto y justifica qué tareas mostrarían sus resultados.

EVIDENCIA PARA LA SEMANA 13

Mapa de subcaracterística → criterio → tarea → evidencia → limitación.

CAPÍTULO 10 / SEMANA 12

Aprendibilidad y protocolos con usuarios

LO QUE CONSEGUIRÁS

Diseñar una sesión pequeña, ética y reproducible basada en tareas reales.

10.1 Por qué el equipo no debe evaluarse a sí mismo

Quien desarrolló la interfaz conoce rutas, controles y errores frecuentes. Esa familiaridad impide apreciar dificultades de principiantes. Por eso conviene observar **participantes voluntarios representativos**, con instrucciones equivalentes, en tareas vinculadas a necesidades reales. Un grupo pequeño puede revelar problemas, pero sus cifras no representan estadísticamente a toda la población.

10.2 Protocolo práctico de evaluación

Elemento	Ejemplo para gestión de citas
Participantes	Cinco voluntarios sin experiencia previa con el prototipo.
Tarea 1	Reservar una cita para un día y horario indicados.
Tarea 2	Corregir una fecha inválida sin reiniciar el formulario.
Éxito	Terminar la tarea con el resultado correcto, sin intervención del evaluador.
Tiempo	Medir segundos desde inicio de consigna hasta conclusión.
Errores	Conteo definido: clic incorrecto, entrada inválida, intento duplicado.
Ayuda	Registrar ayuda solicitada y ayuda proporcionada. No ocultarla.

Antes, durante y después

1. Preparar una consigna neutral, un perfil de tarea, consentimiento simple y datos ficticios.
2. Pedir a la persona que intente completar la tarea; no enseñar previamente la ruta correcta.
3. Observar pasos, tiempo, errores y solicitudes de ayuda, sin grabar información personal innecesaria.
4. Al terminar, preguntar qué le confundió y qué sugeriría mejorar.
5. Consolidar resultados por participante y proponer una corrección verificable.
6. Aplicar una segunda prueba, aclarando si son los mismos participantes (efecto aprendizaje) u otros.

10.3 Tres indicadores simples y sus límites

Éxito por tarea: éxitos sin ayuda / intentos válidos × 100. **Tiempo de tarea:** mediana de segundos entre participantes. **Errores por tarea:** total de errores observados / tareas intentadas, según definición previa. Son medidas didácticas; reporta tamaños muestrales, apoyos y condiciones, no “usabilidad ISO = 90 %”.

Ejemplo ficticio: cuatro de cinco usuarios reservaron sin ayuda. Éxito observado: 4/5 = 80 %. Una persona no completó la tarea. La conclusión correcta es que el prototipo mostró una dificultad para al menos un usuario de este grupo, no que “80 % de todos los usuarios podrán usarlo”.

ÉTICA DEL ENSAYO

Usa participantes voluntarios y seudónimos P01–P05; informa que se evalúa el producto, no la capacidad de la persona. No incluyas diagnósticos, datos de salud, documentos ni videos

identificables sin autorización adecuada.

Comprueba que aprendiste

1. ¿Por qué entrevistar solo a programadores podría sesgar la prueba?
2. ¿Qué información debe registrar el observador si presta ayuda?

APLÍCALO A TU PROYECTO

Planifica dos tareas con cinco voluntarios o tantos como puedas reclutar éticamente y justifica el tamaño real.

EVIDENCIA PARA LA SEMANA 13

Protocolo, hoja por participante, resultados por tarea, problemas hallados y decisiones.

CAPÍTULO 11 / SEMANA 12

Protección contra errores y asistencia al usuario

LO QUE CONSEGUIRÁS

Rediseñar controles y mensajes con criterios verificables y prácticas de accesibilidad.

11.1 Prevenir es mejor que culpar

Una mala interfaz entrega “Error 400” y obliga al usuario a comenzar de nuevo. Una mejor interfaz valida el campo, explica la causa, propone un arreglo y conserva entradas válidas. También previene acciones repetidas deshabilitando temporalmente el envío, confirma operaciones delicadas y ofrece formas de retroceder cuando corresponde.

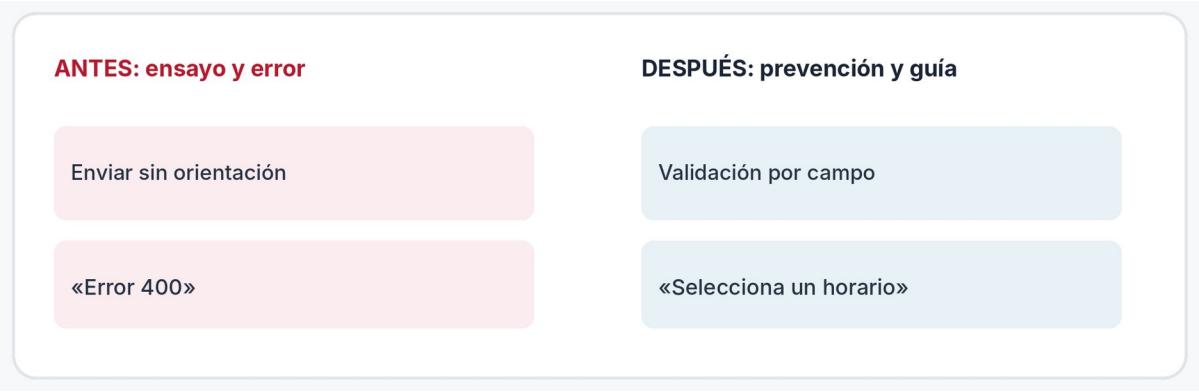


Figura. Comparación didáctica de mensajes genéricos frente a prevención y ayuda contextual.

Problema	Diseño de mejora	Prueba observable
Fecha vacía	Etiqueta clara, campo obligatorio y mensaje específico.	El usuario identifica y corrige la fecha.
Doble envío	Botón deshabilitado durante la solicitud.	Un clic repetido no produce duplicados.
Servicio externo caído	Mensaje de indisponibilidad y alternativa.	La persona comprende que puede reintentar después.
Control sin nombre	Label visible asociado y foco distinguible.	Se opera con teclado y lector compatible.

11.2 Mensajes que enseñan y no castigan

Modelo débil: “Falló la operación”. Modelo orientador: **“No se confirmó la cita porque el horario ya está ocupado. Selecciona otro horario; conservaré el nombre y la fecha que ingresaste.”** Evita textos agresivos, códigos internos para usuarios no técnicos y mensajes sin acción posible.

11.3 Reglas de evaluación de accesibilidad

La **WCAG 2.2 del W3C** proporciona pautas útiles, complementarias a ISO/IEC 25010:2023. Al evaluar formularios, presta atención a etiquetas/instrucciones, identificación de errores, sugerencias de corrección y navegación por teclado. No basta afirmar “mi web es accesible” porque tenga buen contraste; las comprobaciones y tecnologías asistivas utilizadas deben documentarse.

```
<label for="fecha">Fecha de la cita</label>
<input id="fecha" name="fecha" type="date" required
       aria-describedby="ayuda-fecha">
<small id="ayuda-fecha">Elige una fecha válida.</small>
<p id="estado" role="status" aria-live="polite"></p>
```

11.4 Actividad de auditoría en parejas

Primero diagnosticar, después corregir

1. Abre el formulario de tu aplicación y realiza una tarea sin usar el mouse.
2. Enumera tres lugares donde una persona podría cometer un error o necesitar instrucciones.
3. Escribe mensajes específicos de prevención y recuperación.
4. Implementa al menos dos mejoras y registra los fragmentos modificados.
5. Pide a otra persona que repita la tarea y documenta si los obstáculos persisten.

Comprueba que aprendiste

1. ¿Una alerta de JavaScript que desaparece constituye suficiente asistencia contextual?
2. ¿Qué diferencia hay entre un error técnico y un mensaje dirigido al usuario?

APLÍCALO A TU PROYECTO

Mejora tres mensajes del sistema: campo inválido, conflicto de negocio y fallo externo.

EVIDENCIA PARA LA SEMANA 13

Capturas antes/después, fragmentos modificados y tabla de resultados observados.

CAPÍTULO 12 / SEMANA 12

Involucración, inclusión y autodescriptividad

LO QUE CONSEGUIRÁS

Evaluar la interacción sin confundir satisfacción, accesibilidad y retención comercial.

12.1 User engagement con un propósito legítimo

En ISO/IEC 25010:2023, la involucración del usuario se refiere a características que hacen invitante y motivadora la interacción. En un sistema académico o administrativo, eso puede significar información clara, consistencia y retroalimentación positiva. **No equivale a maximizar horas de uso** ni a diseñar mecanismos que distraigan a la persona de su objetivo.

12.2 Cómo estudiarlo con prudencia

Se puede combinar observación y una pregunta postarea: "La secuencia para reservar me resultó clara" con escala de 1 (muy en desacuerdo) a 5 (muy de acuerdo). Es un **ítem didáctico construido para el curso**, no una escala psicométrica validada. Compara respuestas con evidencia de completitud de tareas y evita concluir satisfacción general con muestras pequeñas.

12.3 Inclusividad y autodescriptividad

Diseña etiquetas comprensibles para personas con distintos niveles de experiencia, brinda alternativas de entrada cuando corresponda y evita depender solamente del color. Revisa instrucciones visibles, tamaño de controles, orden del foco, lenguaje sencillo y ejemplos claros. Asegúrate de especificar qué perfil de usuarios probaste y cuáles quedaron fuera de tu alcance.

Foco	Pregunta de observación	Evidencia útil
Involucración	¿La persona mantiene disposición a terminar la tarea?	Observación contextual y preguntas finales.
Inclusividad	¿Hay barreras previsibles para perfiles diversos?	Prueba de teclado, idioma y análisis de perfiles.
Autodescriptividad	¿Se entiende para qué sirve cada control?	Tareas sin instrucciones adicionales y pruebas de etiquetas.

NO MAQUILLES RESULTADOS

Presenta respuestas favorables y críticas por igual. Si dos participantes se confundieron en el mismo paso, esa observación vale más que una frase genérica sobre que "todos quedaron satisfechos".

Comprueba que aprendiste

1. ¿Qué diferencia hay entre involucración y tiempo de permanencia?
2. ¿Puede una interfaz vistosa resultar poco inclusiva?

APLÍCALO A TU PROYECTO

Haz una revisión rápida de accesibilidad y recoge tres comentarios de participantes después de completar una tarea.

EVIDENCIA PARA LA SEMANA 13

Registro de observaciones, mejoras priorizadas y justificación de alcance.

CAPÍTULO 13 / SEMANA 12

Laboratorio de interacción y mejora de interfaz

LO QUE CONSEGUIRÁS

Planear y ejecutar un ciclo de evaluación → rediseño → reevaluación documentado.

13.1 Localiza una línea base

En el laboratorio abre la interfaz web alojada en `http://127.0.0.1:5000/`. Identifica los elementos: nombre ficticio, fecha, horario, botón de confirmar, mensajes de validación y ayuda contextual. No supongas que todos los usuarios comprenden lo mismo: somete las tareas a observación.

Práctica de semana 12

1. Construye una tabla con dos tareas: reservar una cita válida y recuperarse de un horario ocupado.
2. Define las condiciones del participante, las instrucciones y el criterio de éxito antes de la sesión.
3. Registra para cada participante el éxito, tiempo, errores, ayuda y observaciones; usa códigos P01, P02, etc.
4. Selecciona un problema frecuente o de alta severidad para corregir; conserva la captura de “antes”.
5. Edita `public/app.js`, `public/index.html` o `public/estilos.css` para incorporar prevención, ayuda o claridad.
6. Repite el ensayo y presenta resultados comparables, aclarando el posible efecto aprendizaje.
7. Archiva versiones, capturas, matrices, instrucciones y comentario de limitaciones.

13.2 Tabla de registro de participantes (modelo)

Participante	Tarea	Éxito sin ayuda	Tiempo (s)	Errores	Observación
P01	Reservar cita	[sí/no]	[dato]	[n]	[qué ocurrió]
P02	Reservar cita	[sí/no]	[dato]	[n]	[qué ocurrió]
P03	Resolver error	[sí/no]	[dato]	[n]	[qué ocurrió]
P04	Resolver error	[sí/no]	[dato]	[n]	[qué ocurrió]
P05	Resolver error	[sí/no]	[dato]	[n]	[qué ocurrió]

13.3 Interpretación de ejemplo: no escondas aprendizaje

Datos inventados: en la primera ronda, tres de cinco participantes terminaron sin ayuda; tras rediseñar los mensajes, cinco de cinco lo hicieron. La diferencia descriptiva es de 40 puntos porcentuales. Si fueron los mismos cinco usuarios, parte de la mejora podría deberse a conocer ya la tarea. La conclusión debe mencionar ese factor; para contrastar con mayor rigor convendría otro grupo o tareas equivalentes.

Comprueba que aprendiste

1. ¿Qué se debe mantener igual entre el antes y el después?
2. ¿Qué sesgo surge al utilizar los mismos cinco usuarios sin control?

APLÍCALO A TU PROYECTO

Implementa una mejora de interacción y susténtala con al menos una prueba antes/después.

EVIDENCIA PARA LA SEMANA 13

Protocolo, tablas crudas, captura comparativa, cambio de código, interpretación y limitaciones.

PARTE IV | SEMANA 13

APF3: demostrar, informar y defender

Integración de las tres características de calidad y despliegue de la versión 2.

UNIDAD 3 · CALIDAD DE SOFTWARE

PREGUNTA GUÍA

¿Podemos defender la calidad del producto con criterios, pruebas, resultados, evidencia y una versión 2 desplegada?

CAPÍTULO 14 / SEMANA 13

El APF3 y sus productos obligatorios

LO QUE CONSEGUIRÁS

Traducir los requisitos del sílabo a una carpeta de entrega completa y trazable.

14.1 Qué corresponde oficialmente a la semana 13

El cronograma establece para la semana 13 la **recapitulación**, la presentación de **evidencias de prácticas de calidad orientadas a funcionalidad, compatibilidad e interacción**, el **despliegue de la versión 2** de la solución y la evaluación grupal **Avance de Proyecto Final 3 (APF3)**. El sílabo no fija una cantidad obligatoria de páginas ni esta rúbrica detallada; las siguientes pautas se proponen para ordenar una entrega sólida.

14.2 Las cuatro piezas del expediente

Producto	Qué debe mostrar
A · Informe técnico	Problema, alcance, método, requisitos, pruebas, resultados, hallazgos, mejora y conclusiones.
B · Evidencias	Capturas, logs, colecciones, registros, datos y fuentes identificadas por ID.
C · Código y versión 2	Repositorio/commit, cambios efectuados, instructivo de ejecución y despliegue verificable.
D · Sustentación	Demostración del flujo principal y explicación de errores, limitaciones y decisiones.

14.3 Versiones y continuidad del proyecto integrador

La semana 9 considera un despliegue a producción versión 1 dentro del curso. El APF3 pide **versión 2** con las prácticas de calidad incorporadas. No confundas “tener una URL accesible” con demostrar calidad: registra el entorno, el identificador de versión, los cambios respecto a v1 y evidencia de que los requisitos continúan funcionando.

REGLA DE LA SEMANA 13

Toda afirmación importante del informe debe enlazar al menos una evidencia. Si falta una característica completa (por ejemplo, interacción), el informe aún no representa toda la Unidad 3.

14.4 Estructura de carpetas recomendada

```
APF3_GRUPO_XX/
  01_informe/informe_APF3.pdf
  02_funcionalidad/CP-F_*.pdf
  03_compatibilidad/CP-I_*.pdf  EXP-C_*.csv
  04_interaccion/PROTOCOLO_UX.pdf  UX_*.csv
  05_codigo/README.md  version.txt
  06_despliegue/capturas_v2/
  07_anexos/matriz_trazabilidad.xlsx
```

Comprueba que aprendiste

1. ¿Qué pide el sílabo que no queda cubierto con pruebas de funcionalidad y API?
2. ¿Por qué el despliegue debe vincularse con una versión y fecha?

APLÍCALO A TU PROYECTO

Crea la carpeta APF3 de tu grupo, asigna responsables y prepara el inventario de evidencias.

EVIDENCIA PARA LA SEMANA 13

Árbol de archivos, matriz de entregables y registro de versión 2.

CAPÍTULO 15 / SEMANA 13

Metodología, indicadores y decisión de calidad

LO QUE CONSEGUIRÁS

Diseñar una evaluación integrada con umbrales ex ante, mediciones y limitaciones.

15.1 Formula tres preguntas de evaluación

Q1 Funcionalidad: ¿las funciones comprometidas se comportan como el negocio espera? **Q2 Compatibilidad:** ¿se intercambia y utiliza información con terceros, y el sistema se mantiene aceptable al compartir recursos? **Q3 Interacción:** ¿usuarios concretos pueden aprender y finalizar tareas y recuperarse de errores? Para cada pregunta establece indicadores, medios de prueba, resultados y riesgos de validez.

15.2 Matriz de operacionalización propuesta

Dimensión	Indicador / procedimiento	Criterio ilustrativo previo	Evidencia
Completitud	RF verificados / RF comprometidos.	100 % de RF críticos; resto justificado.	Matriz trazabilidad.
Corrección	Pruebas aprobadas / ejecutadas.	≥ 90 % y cero defectos críticos abiertos.	Salida de pruebas.
Interoperabilidad	Contratos y escenarios correctos.	Todos los escenarios críticos pasan.	Postman + logs.
Coexistencia	Δ% de P95 A vs. B.	Degradación ≤ 15 % para carga definida.	Datos crudos y entorno.
Aprendibilidad	Éxitos sin ayuda / intentos.	≥ 4 de 5 en tarea prioritaria.	Protocolo de usuarios.
Protección / asistencia	Errores previstos y recuperables.	Ningún error frecuente sin mensaje accionable.	Inspección + tareas.
Involucración	Pregunta postarea y comentarios.	Interpretación cualitativa, no corte universal.	Cuestionario propio.

Los números anteriores son **ejemplos de criterios didácticos, no umbrales ISO ni rúbrica oficial UTP**. Pueden modificarse justificadamente antes del estudio. No promedies indiscriminadamente porcentajes heterogéneos: el 90 % de pruebas, el 80 % de éxito de tareas y un 15 % de degradación no representan unidades equivalentes.

15.3 Cómo decidir ante evidencia contradictoria

Si la funcionalidad pasa el 95 % de tests pero existe un defecto crítico que permite duplicar una operación de pago, no conviene declarar el sistema aceptable basándose solo en el promedio. Evalúa severidad, importancia del requisito, alcance de la evidencia y riesgos de la solución. Separa **medida**, **interpretación** y **decisión** con una justificación explícita.

15.4 Validez y transparencia

- Distingue **cobertura de requisitos** de **tasa de pruebas aprobadas** y de **proporción de usuarios exitosos**.
- Indica si los datos son simulados, piloto, reales, completos o incompletos.
- Las capturas deben coincidir con la versión, el caso de prueba y el resultado del registro.

- Un repositorio y una URL ayudan a reproducir; no sustituyen por sí solos el análisis.

Comprueba que aprendiste

1. ¿Por qué no corresponde sumar porcentajes de tres dimensiones y dividirlos entre tres?
2. ¿Qué harías si la media parece adecuada, pero una prueba crítica falla?

APLÍCALO A TU PROYECTO

Crea tu matriz operacional de tres características, fija criterios y preséntala antes de correr los ensayos.

EVIDENCIA PARA LA SEMANA 13

Matriz de indicadores con fórmula, fuente, criterio previo, resultado y límites.

CAPÍTULO 16 / SEMANA 13

Redacción completa del informe técnico APF3

LO QUE CONSEGUIRÁS

Seguir una estructura con profundidad, contexto, procedimientos verificables y anexos conectados.

16.1 Índice maestro recomendado para el informe

N.º	Sección de informe	Qué debe responder
Prelim.	Portada, control de versiones, índice y resumen ejecutivo.	¿Quién hizo qué, qué se evaluó y qué se obtuvo?
1	Introducción y problema organizacional.	¿Qué problema real resuelve el producto?
2	Descripción del sistema, alcance y arquitectura.	¿Cuál es la versión y dónde se ejecuta?
3	Marco normativo y criterios ISO/IEC 25010.	¿Qué tres características y subcaracterísticas se evalúan?
4	Requisitos, criterios y trazabilidad.	¿Qué compromiso de calidad existe?
5	Metodología, diseño de pruebas y participantes.	¿Cómo puede replicarse la evaluación?
6	Resultados de semana 10: funcionalidad.	¿Qué se probó, pasó, falló y quedó pendiente?
7	Resultados de semana 11: compatibilidad.	¿Qué demuestra intercambio, uso de datos y coexistencia?
8	Resultados de semana 12: interacción.	¿Qué tareas realizaron usuarios y qué dificultades tuvieron?
9	Hallazgos, mejoras, versión 2 y despliegue.	¿Qué cambió y con qué evidencia?
10	Discusión, límites, conclusiones y próximos pasos.	¿Qué se puede afirmar sin exagerar?
Final	Referencias y anexos (RF/CP/EV, logs, protocolos).	¿Dónde se revisan los datos originales?

16.2 Cómo escribir bien cada parte

Resumen ejecutivo: de 150 a 250 palabras como propuesta editorial, con propósito, versión, tres dimensiones, metodología, hallazgo principal y limitación. Se redacta al final aunque aparezca al inicio. **Introducción:** presenta el problema organizacional y evita comenzar con dos páginas de definiciones genéricas. **Arquitectura:** incluye componentes, dependencias y alcance, sin diagramas ilegibles.

Metodología: especifica qué se midió, quién lo hizo, cuándo, con qué herramientas y criterios previamente definidos. **Resultados:** tablas, porcentajes y gráficos, siempre vinculados con evidencia original. **Discusión:** explica discrepancias, amenazas a la validez y relevancia para el negocio. **Conclusiones:** responde preguntas de investigación/evaluación, sin prometer cumplimiento total de la norma.

16.3 Modelo de redacción “mala → buena”

Redacción débil	Redacción verificable
“Se cumplió la ISO al 100 %”.	“Se comprobaron 7 de 8 RF acordados y 11 de 12 casos ejecutados; persiste RF-08 sin verificar (Anexo A, EV-F-04)”.
“Nuestra API funciona perfecto”.	“La integración pasó 4 de 5 escenarios; la caída del proveedor produjo 503 controlado, pero hubo un contrato incompleto (CP-I05)”.

Redacción débil	Redacción verificable
“La interfaz es amigable”.	“4/5 participantes completaron la tarea sin ayuda; tres señalaron dudas al elegir horario (Tabla UX-02)”.

16.4 Control de integridad de referencias y anexos

Cita el documento oficial ISO, el sílabo y otras fuentes consultadas. No presentes este manual como si fuera el texto íntegro de la norma. Cada imagen debe tener pie y fuente; cada tabla debe aclarar si los datos provienen del software real o de ejemplos educativos. No incluyas capturas artificiales como ejecución real.

PLANTILLA DE PÁRRAFO TÉCNICO

“Para evaluar [subcaracterística] de [módulo y versión], se ejecutaron [N] pruebas diseñadas a partir de [RF]. En el entorno [especificar] se observó [resultado], respecto del criterio previo [umbral/condición]. Las evidencias se registran en [IDs]. La conclusión se limita a [alcance], dado que [limitación]”.

Comprueba que aprendiste

1. ¿Qué apartados deben explicar resultados y cuáles interpretaciones?
2. ¿Qué información debe tener una captura para considerarse trazable?

APLÍCALO A TU PROYECTO

Completa el índice de informe con al menos un párrafo o tabla real en cada sección principal.

EVIDENCIA PARA LA SEMANA 13

Primer borrador integral del informe APF3 y lista de anexos faltantes.

CAPÍTULO 17 / SEMANA 13

Miniinforme resuelto: caso ficticio de citas

LO QUE CONSEGUIRÁS

Leer un modelo consistente que una dimensiones, resultados, mejoras y conclusiones sin inventar conformidad.

ADVERTENCIA METODOLÓGICA

Todos los números de este capítulo son ficticios. Son un ejemplo de cómo razonar y escribir. Nunca deben incorporarse como resultados reales de un proyecto estudiantil.

17.1 Objetivo, versión y alcance

Se evaluó el prototipo **CitasLab v2.0**, una solución de demostración integrada con un proveedor local de horarios. El alcance comprendió las funciones registrar cita, rechazar duplicados y consultar disponibilidad; no abarcó autenticación, persistencia permanente, gestión clínica ni escalabilidad. El análisis se concentró en tres características de producto previstas para la Unidad 3.

17.2 Plan y resultados simulados de funcionalidad

Variable	Dato inventado	Interpretación
RF comprometidos	8	Denominador acordado del ejemplo.
RF verificados	7	Cobertura = 7/8 = 87,5 %.
Pruebas ejecutadas	12	11 aprobadas, 1 fallida, 0 bloqueadas.
Resultado clave	CP-F07 falló	Una reserva duplicada fue creada incorrectamente.

Interpretación: el producto mostró buena ejecución de gran parte del alcance, pero **no puede declararse correcto en el flujo de reservas** mientras persista el defecto crítico. El 91,7 % de tests aprobados no neutraliza ese riesgo.

17.3 Interoperabilidad y coexistencia simuladas

Indicador	Resultado inventado	Conclusión limitada
Escenarios de integración	4/5 correctos	El caso de contrato sin campo horarios produjo fallo.
Proveedor interrumpido	503 con mensaje controlado	Tratamiento de indisponibilidad observado.
P95 sin carga	220 ms	Línea base ficticia.
P95 con carga	264 ms	20 % de degradación en condiciones ficticias.

El umbral académico definido para coexistencia en el caso era ≤ 15 %; por tanto, **el resultado observado no cumplió el criterio propuesto**. Se recomienda repetir con cargas y rondas controladas antes de generalizarlo a otros entornos.

17.4 Capacidad de interacción simulada

Actividad	Dato inventado	Hallazgo
Reservar cita sin ayuda	4/5 participantes	Una persona no identificó el selector de horario.
Recuperar un error	3/5 participantes	Mensajes genéricos generaron confusión.

Actividad	Dato inventado	Hallazgo
Entrevista final	3/5 pidieron ejemplos	Se priorizó ayuda contextual.
Prueba posterior de tarea	5/5 terminaron	Posible efecto aprendizaje: mismos participantes.

17.5 Plan de mejora y despliegue v2

Se corrigió el conflicto de duplicados, se validó la estructura JSON y se mejoraron las etiquetas del formulario. Cada corrección requirió nueva ejecución del caso fallido y de regresiones relevantes. La versión v2.0 se desplegó en un entorno de demostración con identificador de commit, instrucciones de ejecución y capturas fechadas. **Se debe presentar evidencia real de estas acciones en cualquier entrega auténtica.**

17.6 Conclusión modelo, con límites

“La evaluación del caso ficticio encontró avances funcionales, pero también una brecha de cobertura y un defecto crítico; la interoperabilidad necesitó reforzar la validación del contrato, mientras que la coexistencia excedió el límite experimental elegido. Las pruebas exploratorias con cinco usuarios mostraron dificultades de recuperación ante errores. Se implementaron ajustes en v2.0 y se planificó la reevaluación. Por las limitaciones del ambiente y muestra, no se declara conformidad global con ISO/IEC 25010:2023”.

Comprueba que aprendiste

1. ¿Cuál fue el fallo más crítico y por qué el porcentaje global lo podría ocultar?
2. ¿Qué conclusión sería impropia sobre los cinco participantes?

APLÍCALO A TU PROYECTO

Redacta un resumen ejecutivo de 180–220 palabras con tus datos reales. Debe informar al menos un límite.

EVIDENCIA PARA LA SEMANA 13

Borrador de resumen, tres tablas de resultados y conclusiones ligadas a datos propios.

CAPÍTULO 18 / SEMANA 13

Sustentación, rúbrica y checklist final

LO QUE CONSEGUIRÁS

Presentar el APF3 de forma profesional, reconocer límites y preparar la mejora continua.

18.1 Guion de sustentación de 10 minutos (propuesto)

Tiempo	Contenido	Evidencia visual
0:00–1:00	Problema, usuarios, sistema y versión.	Diagrama + URL/commit.
1:00–3:00	Adecuación funcional y un test negativo.	RF, CP, resultado, defecto.
3:00–5:00	Interoperabilidad + coexistencia.	API/contrato + comparación A/B.
5:00–7:00	Capacidad de interacción y cambio de UX.	Tareas, antes/después, errores.
7:00–9:00	Versión 2 y discusión de hallazgos.	Demostración y correcciones.
9:00–10:00	Conclusiones y próximas acciones.	Tres conclusiones y límites.

18.2 Rúbrica referencial propuesta (20 puntos)

Criterio orientativo	Puntos	Máxima evidencia esperada
Funcionalidad	4	RF y criterios claros; pruebas positivas/negativas; defectos.
Interoperabilidad	3	Contrato, escenarios válidos y fallas controladas.
Coexistencia	2	Diseño A/B y discusión de condiciones.
Interacción	4	Tareas con usuarios, prevención/ayuda y mejora.
Informe y trazabilidad	4	Coherencia, análisis, datos, referencias, anexos.
Versión 2 y defensa	3	Demo reproducible, mejoras verificadas, honestidad técnica.

Esta rúbrica es una propuesta docente para autoevaluarse; el sílabo proporcionado solo especifica que APF3 es una evaluación grupal en la semana 13 y representa 20 % de la nota final del curso. Prevalecen las indicaciones oficiales del docente o aula virtual.

18.3 Checklist de diez preguntas antes de entregar

1. ¿La portada identifica grupo, curso, versión, fecha e integrantes?
2. ¿Los requisitos incluyen criterios y vínculos a sus pruebas?
3. ¿Se distinguen cobertura, aprobación de pruebas y tareas de usuarios?
4. ¿Se documentan resultados fallidos y bloqueados sin ocultarlos?
5. ¿Hay intercambio real de datos y manejo de errores de integración?
6. ¿La coexistencia compara un entorno A con un entorno B replicable?
7. ¿Las pruebas de interacción incluyen participantes, tareas y limitaciones?
8. ¿Los cambios de la versión 2 están vinculados a hallazgos y regresión?
9. ¿Las capturas y archivos están ordenados con IDs y son verificables?
10. ¿La discusión y las conclusiones evitan proclamar “certificación ISO”?

18.4 Preguntas difíciles que sí pueden aparecer

- ¿Cómo sabes que el número de pruebas es suficiente para tus requisitos críticos?
- ¿Qué harías si la API entrega 200 pero el campo `horarios` no existe?
- ¿Por qué la mejora del 60 % al 100 % con cinco usuarios no permite generalizar?
- ¿Cómo distingues coexistencia de eficiencia del desempeño en una auditoría?
- ¿Qué resultados cambiarían tu decisión de liberar la versión 2?

Comprueba que aprendiste

1. ¿Qué afirmación del equipo sería la más difícil de defender con evidencia actual?
2. ¿Qué prueba o dato faltante representa el mayor riesgo?

APLÍCALO A TU PROYECTO

Ensaya la defensa entre equipos: uno presenta y otro solicita evidencias precisas. Registra las correcciones antes de entregar.

EVIDENCIA PARA LA SEMANA 13

Acta de revisión cruzada, versión final del informe, anexos y demo de v2.

ANEXOS · Instrumentos de trabajo editables

Copie estas tablas a su informe o a una hoja de cálculo. Los valores entre corchetes deben reemplazarse por datos propios. Los anexos no deben presentarse vacíos en el APF3.

Anexo A. Ficha de identificación del proyecto

Campo	Contenido a completar
Proyecto / sigla	[Complete aquí]
Organización / problema	[Complete aquí]
Grupo / integrantes	[Complete aquí]
Docente / curso / sección	[Complete aquí]
Stack y repositorio	[Complete aquí]
Versión evaluada / fecha	[Complete aquí]
Servidor / URL / entorno	[Complete aquí]
Responsable de pruebas	[Complete aquí]

Anexo B. Matriz de trazabilidad RF–CP–EV

RF / prioridad	Criterio de aceptación	CP vinculado	Resultado	Evidencia / estado
[RF-01]	[Condición medible]	[CP-F01]	[AP/FAL/BLOQ]	[EV-F01]
[RF-02]	[Condición medible]	[CP-F02]	[AP/FAL/BLOQ]	[EV-F02]
[RF-03]	[Condición medible]	[CP-F03]	[AP/FAL/BLOQ]	[EV-F03]

Anexo C. Ficha completa de caso de prueba

Elemento	Registro
Código CP / requisito RF	[Complete aquí]
Característica ISO / subcaracterística	[Complete aquí]
Objetivo y riesgo	[Complete aquí]
Entorno, sistema y commit	[Complete aquí]
Precondición	[Complete aquí]
Datos de entrada	[Complete aquí]
Pasos numerados	[Complete aquí]
Resultado esperado	[Complete aquí]
Resultado obtenido	[Complete aquí]
Estado, fecha y responsable	[Complete aquí]
URL, hash o captura de evidencia	[Complete aquí]
Hallazgo / seguimiento	[Complete aquí]

Anexo D. Contrato y pruebas de interoperabilidad

Elemento	Descripción
Sistema origen y destino	[Complete aquí]
Propósito del intercambio	[Complete aquí]

Elemento	Descripción
URL/ruta y método HTTP	[Complete aquí]
Formato y campos JSON	[Complete aquí]
Reglas de validación	[Complete aquí]
Seguridad y datos de prueba	[Complete aquí]
Escenarios 2xx / 4xx / 5xx	[Complete aquí]
Fallo de red / timeout	[Complete aquí]
Evidencia del uso de datos recibidos	[Complete aquí]

Anexo E. Protocolo experimental de coexistencia

Campo	Condición A (base)	Condición B (compartida)
Software y versión	[dato]	[mismo dato]
Hardware / SO / red	[dato]	[mismo dato]
Proceso adicional	Ninguno	[qué carga]
Rondas / N	[N]	[N]
Mediana / P95	[ms]	[ms]
Fallos / solicitudes	[x/N]	[x/N]
Criterio previo	[meta]	[misma meta]
Cambios observados	[registro]	[registro]

Anexo F. Protocolo de interacción y consentimiento

Consigna neutra propuesta: "Vas a reservar una cita con los datos ficticios proporcionados. Intenta hacerlo como lo harías normalmente. Evaluamos el producto, no tu habilidad. Puedes detenerte cuando quieras. Registraremos tiempos, errores y observaciones sin identificarte por nombre".

Participante / tarea	Éxito sin ayuda	Tiempo	Errores	Ayuda / observación
P01 / T1	[sí/no]	[s]	[n]	[texto]
P02 / T1	[sí/no]	[s]	[n]	[texto]
P03 / T1	[sí/no]	[s]	[n]	[texto]
P04 / T1	[sí/no]	[s]	[n]	[texto]
P05 / T1	[sí/no]	[s]	[n]	[texto]

Anexo G. Registro de defectos y mejoras

Hallazgo	Severidad	Causa probable	Corrección / commit	Retest / evidencia
[DEF-01]	[Alta]	[hipótesis]	[cambio]	[CP/EV]
[DEF-02]	[Media]	[hipótesis]	[cambio]	[CP/EV]
[DEF-03]	[Baja]	[hipótesis]	[cambio]	[CP/EV]

Anexo H. Matriz de hallazgos ISO e indicadores

Subcaracterística	Indicador / criterio	Resultado	Decisión	Limitación
Completitud	[C, umbral]	[dato]	[estado]	[texto]

Subcaracterística	Indicador / criterio	Resultado	Decisión	Limitación
Corrección	[CP, meta]	[dato]	[estado]	[texto]
Pertinencia	[tarea]	[dato]	[estado]	[texto]
Interoperabilidad	[escenarios]	[dato]	[estado]	[texto]
Coexistencia	[Δ%, meta]	[dato]	[estado]	[texto]
Aprendibilidad	[éxitos]	[dato]	[estado]	[texto]
Errores / asistencia	[tareas]	[dato]	[estado]	[texto]
Involucración	[ítem]	[dato]	[estado]	[texto]

Anexo I. Registro de evidencia y reproducibilidad

ID	Fuente / nombre	Versión y fecha	Prueba / requisito	Ruta o enlace
EV-F-01	[captura/log]	[v2 · fecha]	[CP-F01]	[ruta]
EV-I-01	[Postman/json]	[v2 · fecha]	[CP-I01]	[ruta]
EV-C-01	[csv/gráfico]	[v2 · fecha]	[EXP-C]	[ruta]
EV-U-01	[tabla usuarios]	[v2 · fecha]	[UX-T1]	[ruta]

Anexo J. Acta de revisión previa al APF3

Revisión	Sí / No	Observaciones / responsable
Funcionalidad y V&V documentadas	[ ]	[ ]
Interoperabilidad y coexistencia	[ ]	[ ]
Capacidad de interacción con usuarios	[ ]	[ ]
Versión 2 accesible y reproducible	[ ]	[ ]
Anexos coherentes e íntegros	[ ]	[ ]
Conclusiones y referencias verificadas	[ ]	[ ]

BANCO DE ACTIVIDADES · Aprende, aplica y argumenta

Actividad 1. Diez preguntas de dominio conceptual

1. ¿Cuál es la diferencia entre modelo de calidad y certificación ISO?
2. ¿Por qué una cobertura del 100 % de funciones no asegura que todas sean correctas?
3. ¿En qué se distingue validación de verificación?
4. ¿Qué ejemplo diferencia interoperabilidad de coexistencia?
5. ¿Por qué la respuesta 503 puede constituir un test aprobado?
6. ¿Por qué un P95 aislado tiene poca fuerza como conclusión?
7. ¿Qué diferencia hay entre aprendibilidad y protección frente a errores?
8. ¿Qué evidencias se necesitan para sostener una mejora de asistencia al usuario?
9. ¿Cuál es el riesgo de usar cinco participantes y afirmar que toda la población tendrá el mismo resultado?
10. ¿Por qué APF3 exige integrar pruebas, mejoras y versión 2?

Actividad 2. Problemas numéricos resueltos

Planteamiento	Resolución y cautela
9 RF verificados de 12 definidos.	Cobertura propuesta = 9/12 × 100 = **75 %**. No informa si son correctos.
17 pruebas pasaron, 2 fallaron, 1 bloqueada.	Ejecución = 19/20 = **95 %**. Aprobación entre ejecutadas = 17/19 ≈ **89,5 %**. Bloqueada por separado.
P95 base 250 ms; P95 compartido 300 ms.	Δ = (300−250)/250 × 100 = **20 %**. Interpretación según meta previa y entorno.
Cinco usuarios; tres completan sin ayuda.	Éxito observado = 3/5 = **60 %**, no proporción poblacional estimada con precisión.

Actividad 3. Retos integradores para equipos

- **Reto A:** diseña una prueba que descubra un error de resultado aunque el código HTTP sea 200.
- **Reto B:** formula un protocolo de coexistencia que controle actualizaciones en segundo plano.
- **Reto C:** redacta tres mensajes útiles para una validación fallida sin exponer información técnica.
- **Reto D:** detecta una conclusión exagerada sobre usuarios y reescríbela con límites.
- **Reto E:** convierte un hallazgo de compatibilidad en una modificación de la versión 2 y una prueba de regresión.

Actividad 4. Examen de conciencia del proyecto

Completa con tu grupo tres enunciados: **(1)** “El hallazgo que más compromete la calidad de nuestro producto es…”; **(2)** “La evidencia que aún no podemos reproducir es…”; **(3)** “Antes de afirmar que el producto está listo, necesitamos demostrar…”

LABORATORIO · Guía de instalación y problemas frecuentes

Instalación paso a paso para Windows

```
cd ruta\del\laboratorio
py -m venv .venv
.venv\Scripts\activate
python -m pip install -r requirements.txt
python -m unittest test_sistema.py -v
# En terminal A: python proveedor.py
# En terminal B: python sistema.py
```

Si PowerShell bloquea la activación, puedes ejecutar directamente `.venv\Scripts\python.exe` seguido del script, sin cambiar políticas de ejecución del equipo. En macOS/Linux la activación es `source .venv/bin/activate`. Comprueba instalación con `python --version` y que las dos terminales utilizan el entorno elegido.

Problemas y soluciones

Síntoma	Posible causa / comprobación
No module named flask	No se instaló requirements en el Python que ejecuta el programa.
Port 5000/5001 in use	Existe otro proceso usando el puerto; detenerlo o editar ambos archivos coherentemente.
503 al consultar horarios	Proveedor apagado, URL incorrecta o no disponible; revisar terminal 5001.
No responde formulario	Revisar consola del navegador y ruta `/static/app.js`; no abrir HTML con file://.
Tests fallan tras una edición	Identificar test afectado, contrastar regla de negocio y revertir cambio experimental.

Archivos del paquete complementario

- `sistema.py`: aplicación principal y control de errores HTTP.
- `proveedor.py`: servicio ficticio de disponibilidad.
- `test_sistema.py`: seis pruebas automatizadas con proveedor simulado.
- `public/index.html`, `app.js`, `estilos.css`: interfaz editable y accesible en lo básico.
- `medir_coexistencia.py`, `carga_suave.py`: medición introductoria A/B local.
- `README.md` y `requirements.txt`: instalación y advertencias de alcance.

RESTRICCIÓN DEL LABORATORIO

El proyecto no tiene autenticación, BD persistente ni controles necesarios para producción. No se debe desplegar públicamente ni utilizar para registrar pacientes, clientes u otros datos personales.

PROMPTS DE ESTUDIO · La IA como tutor, no como fabricante de datos

Los siguientes prompts ayudan a pensar, revisar y aprender. **Nunca** deben utilizarse para inventar resultados, participantes, registros de consola, capturas o evidencias que no existan.

Prompt 1 · Entender el concepto

"Actúa como tutor de ingeniería de software. Explícame [subcaracterística] de ISO/IEC 25010:2023 en tres niveles: analogía, definición técnica y ejemplo de mi proyecto [descripción]. Luego hazme cinco preguntas y corrige mis respuestas sin darme todo de inmediato".

Prompt 2 · Mejorar los requisitos

"Revisa estos requisitos de mi proyecto [pegar RF sin datos sensibles]. Identifica ambigüedad, criterios no observables y riesgos de prueba. Propón reformulaciones y explícame por qué serían verificables. No inventes condiciones de negocio".

Prompt 3 · Diseñar pruebas

"A partir de estos requisitos y reglas reales [texto], ayúdame a generar casos positivos, negativos, límite y dependencia externa. Entrégame matriz ID–entrada–esperado–riesgo. Advierte qué información necesitas confirmar y no inventes resultados obtenidos".

Prompt 4 · Interpretar resultados

"Estos son mis datos originales [tabla sin datos personales], mis criterios previos [criterios] y el entorno [descripción]. Calcula los indicadores y redacta una interpretación con límites. Distingue claramente observación, inferencia y recomendación".

Prompt 5 · Revisar la coherencia del informe

"Analiza este borrador APF3 [texto]. Comprueba si cada conclusión tiene un requisito, prueba y evidencia identificable. Señala porcentajes sin denominador, supuestos no justificados, contradicciones y declaraciones de conformidad ISO que no se sustenten. No añadas pruebas ficticias".

Prompt 6 · Simular una sustentación

"Evalúa mi APF3 como jurado académico. Hazme una pregunta difícil a la vez sobre funcionalidad, interoperabilidad, coexistencia, interacción y versión 2. Después de cada respuesta califica claridad y evidencia; ofrece una repregunta".

GLOSARIO RÁPIDO

Término	Significado en el manual
Adecuación funcional	Capacidad de proporcionar funciones para necesidades y tareas definidas.
Completitud	Alcance de funciones requeridas que están cubiertas.
Corrección	Exactitud de resultados de funciones bajo condiciones definidas.
Pertinencia	Adecuación de funciones para facilitar la tarea.
Verificación	Comprobación del producto frente a su especificación.
Validación	Comprobación de utilidad frente al uso previsto.
Interoperabilidad	Intercambio y uso de información entre productos.
Coexistencia	Funcionamiento al compartir entorno y recursos con otros productos.
Aprendibilidad	Facilidad del usuario para aprender funciones especificadas.
Protección contra errores	Prevención y tratamiento adecuado de errores de operación.
Asistencia al usuario	Ayuda para que personas diversas alcancen sus objetivos.
Involucración	Características que favorecen una interacción motivadora.
Caso de prueba (CP)	Procedimiento con condiciones, datos y resultado esperado.
Evidencia (EV)	Registro verificable que respalda una observación.
Línea base	Condición de referencia para comparar resultados.
P95	Valor de percentil 95 bajo un método declarado.
APF3	Avance de Proyecto Final 3, evaluación grupal de la semana 13.

FUENTES Y LECTURAS PARA PROFUNDIZAR

Los siguientes enlaces conducen a las organizaciones emisoras o a documentación técnica. Son referencias para ampliar la lectura, **no se copia aquí el contenido íntegro de las normas**.

- **Universidad Tecnológica del Perú (2026).** Sílabo Curso Integrador II: Software (100000S12F), ciclo 2026-2. Documento base suministrado para este material: unidades 3 y cronograma de semanas 10–13.
- **ISO/IEC (2023).** ISO/IEC 25010:2023. Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model. https://www.iso.org/standard/78176.html
- **IEC (2023).** Ficha bibliográfica de ISO/IEC 25010:2023. https://webstore.iec.ch/en/publication/90024
- **ISO 25000 Portal.** Descripción divulgativa de las características y subcaracterísticas de ISO 25010. https://www.iso25000.com/index.php/normas-iso-25000/iso-25010
- **W3C (2023).** Web Content Accessibility Guidelines (WCAG) 2.2. https://www.w3.org/TR/wcag/
- **Postman.** Write scripts to test API response data. https://learning.postman.com/docs/tests-and-scripts/write-scripts/test-scripts/
- **OWASP (2023).** API Security Top 10. https://api-security.owasp.org/editions/2023/en/0x11-t10/

Delimitación de fuentes

Los **temas, la secuencia de semanas, la evaluación APF3 y el despliegue v2** proceden del sílabo del curso. La **organización pedagógica, ejemplos, experimentos, métricas, rúbrica e instrumentos** han sido desarrollados para esta guía. El modelo ISO es el marco de referencia conceptual; no se asegura que estas prácticas equivalgan a una auditoría formal, una certificación o una evaluación exhaustiva de todas las características.

Edición académica 2026 · **Revisado y aprobado por el Mg. Ing. Huamani Uriarte Enrique Lee**.